

GEMV 6: Routes and Fabric DSDs, Part I

Contents

- [layout.csl](#)
- [pe_program.csl](#)
- [run.py](#)
- [commands.sh](#)

Continuing from the previous example, we now break up a single GEMV computation among two PEs.

The host program copies **b** into the **y** tensor of the left PE. The left PE also gets the first $N/2$ columns of **A** and the first $N/2$ values of **x**, and the right PE gets the last $N/2$ columns of **A** and last $N/2$ values of **x**.

The left and right PE both increment their local **y** tensors by computing their piece of **Ax**. Then, the left PE sends its result to the right PE, which increments its **y** tensor by the received values.

Last, the host copies **y** from the right PE, and checks that the result is correct.

To send data from the left PE to the right PE, we must specify a route, known as a color. In [layout.csl](#), [@set_color_config](#) specifies that on the left PE, color 0 will receive data, or wavelets, from the compute element (CE) up the RAMP, and transmit them to the EAST. On the right PE, color 0 will receive wavelets from the **WEST**, and then transmit them down the RAMP to the CE. [@set_tile_code](#) passes the ID of this color to [pe_program](#) as a parameter named [send_color](#), and also sets a parameter called [pe_id](#), to differentiate if the program is running on the left or the right PE.

The [send_right](#) function executed on the left PE defines a [fabout_dsd](#) called [out_dsd](#) that sends **M** wavelets along the color route specified by [send_color](#). [out_dsd](#) is used as the destination operand of [@fmovs](#), and [y_dsd](#) as the source operand. Thus, this operation sends the **M** elements accessed by [y_dsd](#) along the fabric as specified by [out_dsd](#).

The [recv_left](#) function executed on the right PE receives the data in a [fabin_dsd](#) named [in_dsd](#), used in an [@fadds](#) operation that increments the **M** elements of **y** on this PE by the **M** received values.

Note that this program also provides an example of a local task. The [@fmovs](#) and [@fadds](#) operations are performed asynchronously; when these operations are done, the color [exit_color](#) is activated, which activates the task [exit_task](#). This task unblocks [memcpy](#)'s command stream, allowing additional commands from the host program to proceed.

Note

See [GEMV Tutorial 6: Routes and Fabric DSDs](#) for a step-by-step walkthrough of this example.

layout.csl

```
// matrix dimensions on each PE
param M: i16;
param N: i16;

// Colors
const send_color: color = @get_color(0); // Color used to send/recv data between PEs

// This example only uses 2 PEs
const memcpy = @import_module("<memcpy/get_params>", .{
    .width = 2,
    .height = 1,
});

layout {
    // PE coordinates are (column, row)
    @set_rectangle(2, 1);

    // Left PE (0, 0)
    @set_tile_code(0, 0, "pe_program.csl", .{
        .memcpy_params = memcpy.get_params(0),
        .M = M,
        .N_per_PE = N / 2,
        .pe_id = 0,
        .send_color = send_color
    });

    // Left PE sends its result to the right
    @set_color_config(0, 0, send_color, .{ .routes = .{ .rx = .{RAMP}, .tx = .{EAST} } });

    // Right PE (1, 0)
    @set_tile_code(1, 0, "pe_program.csl", .{
        .memcpy_params = memcpy.get_params(1),
        .M = M,
        .N_per_PE = N / 2,
        .pe_id = 1,
        .send_color = send_color
    });

    // Right PE receives result of left PE
    @set_color_config(1, 0, send_color, .{ .routes = .{ .rx = .{WEST}, .tx = .{RAMP} } });

    // export symbol names
    @export_name("A", [*]f32, true);
    @export_name("x", [*]f32, true);
    @export_name("y", [*]f32, true);
    @export_name("compute", fn()void);
}
```

pe_program.csl

```

param memcpy_params;

// Matrix dimensions
param M: i16;
param N_per_PE: i16;

// ID of PE (0 is left, 1 is right)
param pe_id: i16;

// Colors
param send_color: color; // Color used to send/recv data between PEs

// Queue IDs
const send_color_oq = @get_output_queue(2);
const send_color_iq = @get_input_queue(2);

// Task ID used by a local task to unblock cmd stream
const exit_task_id: local_task_id = @get_local_task_id(9);

// memcpy module provides infrastructure for copying data
// and launching functions from the host
const sys_mod = @import_module("<memcpy/memcpy>", memcpy_params);

// 48 kB of global memory contain A, x, b, y
var A: [M*N_per_PE]f32; // A is stored column major
var x: [N_per_PE]f32;
var y: [M]f32;

// DSDs for accessing A, b, y
// A_dsd accesses column of A
var A_dsd = @get_dsd(mem1d_dsd, {.base_address = &A, .extent = M });
var y_dsd = @get_dsd(mem1d_dsd, {.base_address = &y, .extent = M });

// ptrs to A, x, b, y will be advertised as symbols to host
var A_ptr: [*]f32 = &A;
var x_ptr: [*]f32 = &x;
var y_ptr: [*]f32 = &y;

// Compute gemv
fn gemv() void {
    // Loop over all columns of A
    for (@range(i16, N_per_PE)) |i| {
        // Calculate contribution to A*x from ith column of A, ith elem of x
        @fmacs(y_dsd, y_dsd, A_dsd, x[i]);
        // Move A_dsd to next column of A
        A_dsd = @increment_dsd_offset(A_dsd, M, f32);
    }
}

fn send_right() void {
    const out_dsd = @get_dsd(fabout_dsd, if (@is_arch("wse3")) {.
        .extent = M, .output_queue = send_color_oq

```

```

        } else {. {
            .fabric_color = send_color, .extent = M,
            .output_queue = send_color_oq
        });
    // After fmovs is done, activate exit_task to unblock cmd_stream
    @fmovs(out_dsd, y_dsd, .{ .async = true, .activate = exit_task_id });
}

fn recv_left() void {
    const in_dsd = @get_dsd(fabin_dsd, .{
        .extent = M,
        .input_queue = send_color_iq
    });
    // After fadds is done, activate exit_task to unblock cmd stream
    @fadds(y_dsd, y_dsd, in_dsd, .{ .async = true, .activate = exit_task_id });
}

// Call gemv function and send/ receive partial result y
fn compute() void {
    gemv();
    if (pe_id == 0) {
        send_right();
    } else {
        recv_left();
    }
}

task exit_task() void {
    sys_mod.unblock_cmd_stream();
}

comptime {
    // When exit_task_id is activated, exit_task will execute
    @bind_local_task(exit_task, exit_task_id);

    @initialize_queue(send_color_oq, if (@is_arch("wse3")) .{ .color = send_color } else .{});
    @initialize_queue(send_color_iq, .{ .color = send_color });

    @export_symbol(A_ptr, "A");
    @export_symbol(x_ptr, "x");
    @export_symbol(y_ptr, "y");
    @export_symbol(compute);
}

```

run.py

```

#!/usr/bin/env cs_python

import argparse
import json
import numpy as np

```

```

from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime, MemcpyDataType, MemcpyOrder #
pylint: disable=no-name-in-module

# Read arguments
parser = argparse.ArgumentParser()
parser.add_argument('--name', help="the test compile output dir")
parser.add_argument('--cmaddr', help="IP:port for CS system")
args = parser.parse_args()

# Get matrix dimensions from compile metadata
with open(f"{args.name}/out.json", encoding='utf-8') as json_file:
    compile_data = json.load(json_file)

# Matrix dimensions
N = int(compile_data['params']['N'])
M = int(compile_data['params']['M'])

# Construct A, x, b
A = np.arange(M*N, dtype=np.float32).reshape(M,N)
x = np.full(shape=N, fill_value=1.0, dtype=np.float32)
b = np.full(shape=M, fill_value=2.0, dtype=np.float32)

# Calculate expected y
y_expected = A@x + b

# Size of N dimension on each PE
N_per_PE = N // 2

# Construct a runner using SdkRuntime
runner = SdkRuntime(args.name, cmaddr=args.cmaddr)

# Get symbols for A, x, y on device
A_symbol = runner.get_id('A')
x_symbol = runner.get_id('x')
y_symbol = runner.get_id('y')

# Load and run the program
runner.load()
runner.run()

# Copy b into y of PE (0, 0)
runner.memcpy_h2d(y_symbol, b, 0, 0, 1, 1, M, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)

# Copy A in column major format
# PE (0, 0) gets first N/2 columns; PE (1, 0) gets last N/2 columns
runner.memcpy_h2d(A_symbol, A.transpose().ravel(), 0, 0, 2, 1, M*N_per_PE, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)

# PE (0, 0) gets first N/2 elements; PE (1, 0) gets last N/2 elements
runner.memcpy_h2d(x_symbol, x, 0, 0, 2, 1, N_per_PE, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)

# Launch the compute function on device
runner.launch('compute', nonblock=False)

```

```

# Copy y back from PE (1, 0)
y_result = np.zeros([M], dtype=np.float32)
runner.memcpy_d2h(y_result, y_symbol, 1, 0, 1, 1, M, streaming=False,
    order=MemcpyOrder.ROW_MAJOR, data_type=MemcpyDataType.MEMCPY_32BIT, nonblock=False)

# Stop the program
runner.stop()

# Ensure that the result matches our expectation
np.testing.assert_allclose(y_result, y_expected, atol=0.01, rtol=0)
print("SUCCESS!")

```

commands.sh

```

#!/usr/bin/env bash

set -e

cslc --arch=wse3 ./layout.csl --fabric-dims=11,3 \
--fabric-offsets=4,1 --params=M:4,N:6 -o out --memcpy --channels 1
cs_python run.py --name out

```